COP3330C Module 13 Programming Assignment 5

In this assignment, you will implement a multithreaded printer simulation that uses the State Design Pattern and thread-safe programming techniques to manage a shared printing system. The application simulates a printer with two primary states: **Idle** and **Printing**. These states work together to process a queue of print jobs retrieved from a centralized document repository. Multiple worker threads will act as clients, retrieving documents from the repository and submitting them to the printer. The printer processes the jobs sequentially, transitioning between states as needed, while simulating the time required to print each job. To ensure proper coordination between threads and avoid race conditions, shared resources—such as the print queue and document repository—must be protected using a ReentrantLock.

The GitHub Classroom repository includes a starter project that implements the printer, printer queue, and document repository using a single worker thread that does not require locks. To complete the assignment, you will replace the single worker thread with three separate worker threads. Additionally, you must incorporate a ReentrantLock and associated lock operations to ensure thread-safe access to the document repository (an ArrayList) and the printer queue (a LinkedList) shared among the worker threads.

Review the starter code and verify it builds and executes successfully. Once you have an executable project, make the following changes:

1. Add the ReentrantLock code to the DocumentRepository class.

Declare the ReentrantLock: Add a private final Lock lock; to the class and initialize it in the constructor using new ReentrantLock().
Lock Critical Sections: Use lock.lock() before accessing shared resources like documentList and documentIndex.
Unlock Safely: Always call lock.unlock() in a finally block to ensure the lock is released, even if an exception occurs.
Protect All Shared Access: Wrap both the getNextDocument() and hasMoreDocuments() methods with locks to prevent race conditions.

Here is an example of using the ReentrantLock locking sequence:

```
lock.lock(); // Acquire the lock
try {
   // Critical section: code that accesses or modifies shared resources
   sharedResource.doSomething();
   System.out.println("Shared resource accessed safely.");
} finally {
   lock.unlock(); // Always release the lock, even if an exception occurs
}
```

2. Add the ReentrantLock code to the Printer class.

Declare the ReentrantLock: Add a private final Lock lock; field to the Printer class and initialize it in the constructor using new ReentrantLock().
Lock Critical Sections: Use lock.lock() before accessing shared resources like the printQueue to ensure thread-safe access.
Unlock Safely: Always call lock.unlock() in a finally block to ensure the lock is released, even if an exception occurs.
Protect the Print Queue:

- **addDocument(String document):** Lock the method to ensure that adding a document to the queue is thread-safe.
- **pollJob():** Lock the method to prevent concurrent threads from simultaneously removing jobs from the queue.
- **hasPendingJobs():** Lock the method to safely check whether the queue is empty.

Protect State Transitions: In the processQueue() method, lock the section where the printer's state is accessed or modified to avoid race conditions during state transitions.
Coordinate Printing Operations:

- Lock the block of code in the print(PrintJob job) method to ensure that only one thread interacts with the printer hardware simulation at a time.

Avoid Deadlocks: Ensure locks are not held for longer than necessary and avoid nested locks to prevent deadlock scenarios.

3. Add the three worker threads in the Printer class's main method.

Create Multiple Worker Instances: Instantiate three separate PrinterWorker objects, each using the same Printer and DocumentRepository objects to simulate concurrent clients accessing shared resources.
Start Threads: Create Thread objects for each PrinterWorker instance and call their start() method to initiate execution.
Pass Common Shared Resources: Ensure all three PrinterWorker instances share the same Printer and DocumentRepository objects to replicate a real-world shared environment.
Join Threads: Use the join() method on each thread to ensure the main program waits for all worker threads to finish execution before exiting.

Submission:

- **Submit your project solution and a screen snip of your output to the GitHub Classroom repository.**